

LISTA DE EXERCÍCIOS - REVISÃO P2

1. Considere as seguintes afirmações sobre estruturas de dados:

- I. Em uma lista encadeada circular o último elemento pode ser encontrado diretamente a partir do primeiro, sem a necessidade de se percorrer toda a lista.
- II. Os elementos de uma lista encadeada simples são armazenados sequencialmente na memória.
- III. Inserções e remoções são mais eficientes em Listas Estáticas quando comparadas a Listas Encadeadas.
- IV. **Pilha** é uma estrutura de dados do tipo FIFO (First-In First-Out)
- V. Os elementos em uma pilha são sempre removidos na mesma ordem em que foram inseridos.
- VI. Em uma árvore binária de busca o filho da direita de um nó é menor que o filho da esquerda.

Assinale a alternativa correta.

- (a) As afirmações I, IV e V estão corretas.
- (b) As afirmações II, III e IV estão corretas.
- (c) As afirmações I, III, IV e VI estão corretas.
- (d) As afirmações II, III, V e IV estão incorretas.
- (e) **Todas as afirmações estão incorretas.**

Obs: a afirmação IV do enunciado estava com erro de digitação, o certo é Pilha e não Fila, de forma que a afirmativa fique incorreta, tornando a alternativa E a resposta correta da questão.

2. (ENADE - 2011) No desenvolvimento de um software que analisa bases de DNA, representadas pelas letras A, C, G, T, utilizou-se as estruturas de dados: pilha e fila. Considere que, se uma sequência representa uma pilha, o topo é o elemento mais à esquerda; e se uma sequência representa uma fila, a sua frente é o elemento mais à esquerda. Analise o seguinte cenário:

*“A sequência inicial ficou armazenada na **primeira estrutura de dados** na seguinte ordem: (A, G, T, C, A, G, T, T). Cada elemento foi retirado da primeira estrutura de dados e inserido na **segunda estrutura de dados**, e a sequência ficou armazenada na seguinte ordem: (T, T, G, A, C, T, G, A). Finalmente, cada elemento foi retirado da segunda estrutura de dados e inserido na **terceira estrutura de dados** e a sequência ficou armazenada na seguinte ordem: (T, T, G, A, C, T, G, A)”.*

Qual a única sequência de estruturas de dados apresentada a seguir pode ter sido usada no cenário acima?

- (a) **Fila - Pilha - Fila.**
- (b) Fila - Fila - Pilha.
- (c) Fila - Pilha - Pilha.
- (d) Pilha - Fila - Pilha.

(e) Pilha - Pilha - Pilha.

3. (Questão adaptada do ENADE CCO 2014: Q16) Uma pilha é uma estrutura de dados que armazena uma coleção de itens de dados relacionados e que garante o seguinte funcionamento: o último elemento a ser inserido é o primeiro a ser removido. É comum na literatura utilizar os nomes *push* e *pop* para as operações de inserção e remoção de um elemento em uma pilha, respectivamente. O seguinte trecho de código em linguagem C define uma estrutura de dados pilha utilizando um vetor de inteiros, bem como algumas funções para sua manipulação e o programa `main()` que utiliza esta pilha.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct pilha {
    int topo;
    int elementos[100];
} Pilha;

Pilha *criaPilha(){
    Pilha *p = malloc(sizeof(Pilha));
    p->topo = -1;
    return p;
}

void push(Pilha *p, int elemento){
    if(p->topo >= 99) return;
    p->elementos[++ p->topo] = elemento;
}

int pop(Pilha *p){
    int a = p->elementos[p->topo];
    p->topo - -;
    return a;
}

int main(){
    Pilha *p = criaPilha();
    push(p, 2);
    push(p, 3);
    push(p, 4);
    pop(p);
    push(p, 2);
    int a=pop(p) + pop(p);
    push(p, a);
    a += pop(p);
    printf(" %d ", a);
    return 0;
}
```

A esse respeito, responda e justifique as seguintes questões:

- (a) Qual a saída do programa e como estará a pilha no final da função *main*?

Saída: 10

Pilha: 2

- (b) O valor exibido pelo programa seria o mesmo caso a instrução `a += pop(p)` fosse trocada por `a += a`?

Sim, pois o número retornado por `pop(p)` é o próprio valor de `a`, que foi empilhado anteriormente.

- (c) Em relação à memória, é opcional chamar a função `free(p)`, pois o vetor usado na pilha é alocado estaticamente?

Não é opcional. Embora o vetor `elementos` seja alocado estaticamente, é necessário desalocar a pilha `p` pois a estrutura utilizada é uma Pilha Estática que é alocada dinamicamente quando se cria a pilha.

4. Seja `No` uma *struct* contendo os ponteiros `esq` e `dir` também do tipo `No`. Considere a seguinte função, que remove um nó (`atual`) de uma árvore binária de busca e retorna o nó que irá substituí-lo (nó mais a direita da subárvore esquerda, se houver filhos à esquerda do nó atual, caso contrário, o nó filho da direita):

```
int remover_no(No *atual) {
    No *no1, *no2;
    if(atual->esq == NULL){
        no2 = atual->dir;
        free(atual);
        return no2; }
    no1 = atual;
    no2 = atual->esq;
    while(no2->dir != NULL){
        no1 = no2;
        no2 = no2->dir;
    }
    if(no1 != atual){
        XXX
        YYY
    }
    ZZZ
    free(atual);
    return no2; }
```

Qual alternativa contém os trechos **XXX**, **YYY** e **ZZZ** para completar corretamente a função?

- (a) `no1->dir = no2->esq;` `no2->esq = atual->dir;` `no2->dir = atual->esq;`
 - (b) `no2->dir = no2->esq;` `no1->esq = atual->esq;` `no1->dir = atual->dir;`
 - (c) `no2->dir = no1->esq;` `no1->esq = atual->esq;` `no2->esq = atual->dir;`
 - (d) **`no1->dir = no2->esq;` `no2->esq = atual->esq;` `no2->dir = atual->dir;`**
 - (e) `no1->dir = no2->dir;` `no2->dir = atual->esq;` `no2->esq = atual->esq;`
5. Seja `No` uma *struct* contendo os ponteiros `esq` e `dir` também do tipo `No`. Considere a seguinte função, que retorna o número de nós em uma árvore binária, dado um ponteiro de ponteiro para a sua raiz:

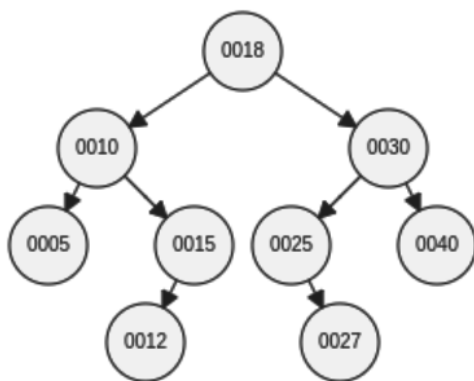
```
int quant_nos(No **raiz) {
    if(raiz == NULL || *raiz == NULL){ return 0; }
    XXX
}
```

Qual alternativa contém o trecho de código **XXX** para completar corretamente a função?

- (a) `return 1 + quant_nos(&raiz->esq) + quant_nos(&raiz->dir);`
- (b) `return 2 * quant_nos(&raiz);`
- (c) `return 1 + quant_nos(&(raiz->esq)) + quant_nos(&(raiz->dir));`
- (d) **`return 1 + quant_nos(&((*raiz)->esq)) + quant_nos(&((*raiz)->dir));`**
- (e) `return quant_nos(&((*raiz)->esq)) + quant_nos(&((*raiz)->dir));`

6. Suponha que T seja uma árvore binária de busca inicialmente vazia, e considere a inserção dos elementos 18, 30, 25, 10, 15, 5, 40, 12 e 27 em T , nesta ordem.

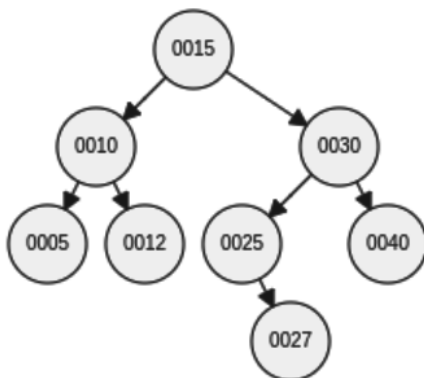
a) Desenhe a árvore obtida após todas as inserções.



b) Assinale a alternativa abaixo que apresenta a sequência correspondente a um percurso da árvore T em **pré-ordem** e **pós-ordem**, respectivamente?

- (a) 18, 10, 5, 15, 12, 30, 25, 27 e 40 // 5, 10, 12, 15, 18, 25, 27, 30 e 40
 (b) 18, 30, 25, 10, 15, 5, 40, 12 e 27 // 5, 12, 15, 10, 27, 25, 40, 30 e 18
 (c) 5, 12, 15, 10, 27, 25, 40, 30 e 18 // 18, 30, 25, 10, 15, 5, 40, 12 e 27
 (d) **18, 10, 5, 15, 12, 30, 25, 27 e 40 // 5, 12, 15, 10, 27, 25, 40, 30 e 18**
 (e) 5, 12, 15, 10, 27, 25, 40, 30 e 18 // 18, 10, 5, 15, 12, 30, 25, 27 e 40

c) Desenhe a árvore obtida após a remoção do elemento 18.



7. Considere uma implementação de **TAD Árvore Binária** que apresenta as seguintes definições:

```

typedef struct noArvore{
    int dado;
    struct noArvore *esq;
    struct noArvore *dir;
}NoArvore;

typedef NoArvore *Arvore;
  
```

Implemente uma função que compare duas **árvores binárias de busca** e verifique se elas são iguais. A função DEVE retornar 1, se as árvores forem iguais e 0, caso contrário. O protótipo da função DEVE ser: `int compararArvore(Arvore *A, Arvore *B)`.

```
int compararArvore(ArvoreBin *A, ArvoreBin *B){

    // Se ambos os nós forem NULL então as subárvores são iguais
    if (*A == NULL && *B == NULL) {
        return 1;
    }

    // Se um dos nós é NULL e o outro não, as subárvores são diferentes
    if(*A == NULL || *B == NULL) {
        return 0;
    }

    // Se os valores dos nós atuais forem diferente, as subárvores são diferentes
    if((*A)->dado != (*B)->dado) {
        return 0;
    }

    // Comparar as subárvores recursivamente
    return compararArvore(&((*A)->dir), &((*B)->dir)) && compararArvore(&((*A)->esq),
&((*B)->esq));

}
```